

Gemini Orchestrator — FastAPI + Next.js

(Complete Starter)

A production-grade starter that unifies **AI Absence** and **SOW Generator** behind a single **session-aware chat UI** powered by **Gemini**. It keeps the conversation **inside chat**, performs **real actions** through typed tool endpoints, and **refuses out-of-scope** requests.

Language stack: **Python FastAPI** (backend) + **Next.js (App Router)** frontend.

Monorepo Layout

```
.
├── backend/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py
│   │   ├── config.py
│   │   ├── models.py
│   │   ├── state.py
│   │   ├── router_rules.py
│   │   ├── orchestrator.py
│   │   └── llm/
│   │       ├── __init__.py
│   │       ├── system_prompt.py
│   │       └── gemini.py
│   └── tools/
│       ├── __init__.py
│       ├── absence.py
│       └── sow.py
├── requirements.txt
├── .env.example
└── frontend/
    ├── app/
    │   ├── api/
    │   │   └── chat/route.ts
    │   └── page.tsx
    ├── package.json
    ├── next.config.mjs
    └── tsconfig.json
```

Backend

backend/app/config.py

```
from pydantic import BaseSettings

class Settings(BaseSettings):
    GEMINI_API_KEY: str
    # Optional infra hooks
    DATABASE_URL: str = "sqlite:///./orchestrator.db"
    REDIS_URL: str = ""
    HOST_ABSENCE_INTERNAL: str = "http://localhost:8000" # if you split
services
    HOST_SOW_INTERNAL: str = "http://localhost:8000"

    class Config:
        env_file = ".env"

settings = Settings()
```

backend/app/models.py

```
from typing import Any, Dict, List, Optional
from pydantic import BaseModel, Field

# Envelope the model must return (strict JSON)
class ToolCall(BaseModel):
    name: str
    args: Dict[str, Any] = {}

class ChatEnvelope(BaseModel):
    mode: str = Field(regex="^(IDLE|ABSENCE|SOW)$")
    intent: Optional[str] = None
    slots: Dict[str, Any] = {}
    tool_call: Optional[ToolCall] = None
    message_to_user: str

# API I/O
class ChatRequest(BaseModel):
    session_id: str
    text: str

class ChatResponse(ChatEnvelope):
    pass
```

```

# Shared typed payloads (tool schemas)
class AbsenceMarkAbsent(BaseModel):
    employee_id: str
    date: str # YYYY-MM-DD
    reason: Optional[str] = None

class AbsenceMarkPresent(BaseModel):
    employee_id: str
    date: str

class SOWStart(BaseModel):
    project_name: str

class ContactInfo(BaseModel):
    name: str
    email: str
    phone: Optional[str] = None
    address: Optional[str] = None

class ServiceItem(BaseModel):
    name: str
    description: str

class DeliverableItem(BaseModel):
    name: str
    description: str

class SOWUpdate(BaseModel):
    sow_id: str
    oracle_rep: ContactInfo
    billing_contact: ContactInfo
    services: List[ServiceItem]
    deliverables: List[DeliverableItem]
    acceptance: str

```

backend/app/state.py

```

from typing import Dict, Any
from collections import defaultdict

# Simple in-memory fallback (replace with Redis or DB for prod)
_sessions: Dict[str, Dict[str, Any]] = defaultdict(dict)

DEFAULT_SNAPSHOT = {
    "mode": "IDLE", # IDLE | ABSENCE | SOW
    "active_sow_id": None, # track active SOW intake id

```

```

        "slots": {},
    }

def get_snapshot(session_id: str) -> Dict[str, Any]:
    snap = _sessions.get(session_id) or {}
    if not snap:
        _sessions[session_id] = {**DEFAULT_SNAPSHOT}
    return _sessions[session_id]

def save_snapshot(session_id: str, data: Dict[str, Any]):
    _sessions[session_id] = data

```

backend/app/router_rules.py

```

def route_domain(user_text: str, current_mode: str) -> str:
    txt = user_text.lower()
    if any(k in txt for k in ["absent", "leave", "attendance", "present"]):
        return "ABSENCE"
    if any(k in txt for k in ["sow", "statement of work", "deliverable",
"milestone"]):
        return "SOW"
    return current_mode if current_mode in ("ABSENCE", "SOW") else "IDLE"

SEED_TESTS = [
    ("mark manju absent today", "ABSENCE"),
    ("is john absent on 2025-10-02?", "ABSENCE"),
    ("generate sow for predictive maintenance", "SOW"),
    ("add deliverables: reporting dashboard", "SOW"),
    ("what's the weather?", "IDLE"),
]

```

backend/app/llm/system_prompt.py

```

SYSTEM_PROMPT = (
    "You are the Orchestrator for TWO capabilities only:\n"
    "1) AI Absence: mark/list/check absences.\n"
    "2) SOW Generator: run structured intake and produce a final SOW doc.\n\n"
    "Rules:\n"
    "- Stay strictly within these domains. If out-of-scope, reply: \"Sorry, I\n"
    "can help only with Absence and SOW generation.\"\n\n"
    "- Prefer ACTIONS via tool calls whenever user implies an operation.\n"
    "- Maintain session mode: IDLE | ABSENCE | SOW. While inside SOW, continue\n"
    "intake until user finishes or cancels.\n"
    "- ALWAYS return a JSON object with fields: mode, intent, slots, tool_call,\n"
    "message_to_user.\n"

```

```
        "- Keep questions brief when collecting missing slots.\n"
    )
```

backend/app/llm/gemini.py

```
import google.generativeai as genai
from app.config import settings
from app.models import ChatEnvelope
from app.llm.system_prompt import SYSTEM_PROMPT

# Configure Gemini
genai.configure(api_key=settings.GEMINI_API_KEY)
MODEL_NAME = "gemini-1.5-pro"

# Strict JSON envelope schema
RESPONSE_SCHEMA = {
    "type": "object",
    "properties": {
        "mode": {"type": "string", "enum": ["IDLE", "ABSENCE", "SOW"]},
        "intent": {"type": ["string", "null"]},
        "slots": {"type": "object"},
        "tool_call": {
            "type": ["object", "null"],
            "properties": {
                "name": {"type": "string"},
                "args": {"type": "object"}
            },
            "required": ["name", "args"]
        },
        "message_to_user": {"type": "string"}
    },
    "required": ["mode", "slots", "message_to_user"],
}

model = genai.GenerativeModel(
    model_name=MODEL_NAME,
    system_instruction=SYSTEM_PROMPT,
)

def build_context(session_snapshot: dict, user_text: str) -> list:
    # Compact conversation snapshot. You can extend with last turns, vector
    # recalls, etc.
    system_state = (
        f"SESSION MODE: {session_snapshot.get('mode', 'IDLE')}\n"
        f"ACTIVE_SOW_ID: {session_snapshot.get('active_sow_id')}\n"
        f"SLOTS: {session_snapshot.get('slots', {})}\n"
    )
```

```

    )
    return [
        {"role": "user", "parts": [
            "You will reply ONLY with the required JSON envelope.",
            "Session snapshot:", system_state,
            "User:", user_text
        ]}
    ]

def ask_gemini(session_snapshot: dict, user_text: str) -> ChatEnvelope:
    contents = build_context(session_snapshot, user_text)
    resp = model.generate_content(
        contents,
        generation_config={
            "response_mime_type": "application/json",
            "response_schema": RESPONSE_SCHEMA,
            "temperature": 0.2,
        },
    )
    data = resp.text
    # google.generativeai returns JSON string for application/json. Pydantic
    parse:
    return ChatEnvelope.model_validate_json(data)

```

backend/app/tools/absence.py

```

from fastapi import APIRouter, HTTPException
from app.models import AbsenceMarkAbsent, AbsenceMarkPresent

router = APIRouter(prefix="/absence", tags=["Absence"])

# TODO: Replace with your real DB writes

_EMPLOYEES = {
    "manju": "E123",
    "john": "E777",
}

_ABSENCES = set() # (employee_id, date)

@router.post("/mark_absent")
def mark_absent(req: AbsenceMarkAbsent):
    key = (req.employee_id, req.date)
    _ABSENCES.add(key)
    return {"ok": True, "message": f"Marked {req.employee_id} absent on {req.date}"}

```

```

@router.post("/mark_present")
def mark_present(req: AbsenceMarkPresent):
    key = (req.employee_id, req.date)
    if key in _ABSENCES:
        _ABSENCES.remove(key)
    return {"ok": True, "message": f"Marked {req.employee_id} present on {req.date}"}

@router.get("/status/{employee_id}")
def get_status(employee_id: str, date: str):
    return {
        "employee_id": employee_id,
        "date": date,
        "status": "absent" if (employee_id, date) in _ABSENCES else "present"
    }

@router.get("/resolve")
def resolve_employee(q: str):
    # Extremely naive resolver – replace with your actual employees DB lookup
    for k, v in _EMPLOYEES.items():
        if q.lower() in k:
            return {"ok": True, "employee_id": v, "name": k}
    raise HTTPException(404, detail="Employee not found")

```

backend/app/tools/sow.py

```

from fastapi import APIRouter
from app.models import SOWStart, SOWUpdate

router = APIRouter(prefix="/sow", tags=["SOW"])

# In-memory SOW sessions (replace with DB)
_SOW = {}

@router.post("/start")
def start(req: SOWStart):
    sow_id = f"SOW{len(_SOW)+1:04d}"
    _SOW[sow_id] = {
        "project_name": req.project_name,
        "oracle_rep": None,
        "billing_contact": None,
        "services": [],
        "deliverables": [],
        "acceptance": None,
        "status": "INTAKE",
    }

```

```

    }
    return {"ok": True, "sow_id": sow_id, "message": f"SOW started for {req.project_name}"}

@router.post("/update")
def update(req: SOWUpdate):
    assert req.sow_id in _SOW, "Invalid SOW id"
    _SOW[req.sow_id].update({
        "oracle_rep": req.oracle_rep.model_dump(),
        "billing_contact": req.billing_contact.model_dump(),
        "services": [s.model_dump() for s in req.services],
        "deliverables": [d.model_dump() for d in req.deliverables],
        "acceptance": req.acceptance,
        "status": "READY_TO_RENDER",
    })
    return {"ok": True, "message": f"SOW {req.sow_id} updated"}

@router.post("/generate/{sow_id}")
def generate(sow_id: str):
    # TODO: merge into DOCX/PDF via your fixed template
    return {"ok": True, "url": f"/files/{sow_id}.docx"}

```

backend/app/orchestrator.py

```

import httpx
from fastapi import APIRouter
from app.models import ChatRequest, ChatResponse, ChatEnvelope
from app.state import get_snapshot, save_snapshot
from app.router_rules import route_domain
from app.llm.gemini import ask_gemini

router = APIRouter(prefix="/orchestrator", tags=["Orchestrator"])

# Tool name → local endpoint (or internal function) mapping
TOOL_MAP = {
    "absence.resolve_employee": ("GET", "/absence/resolve"),
    "absence.mark_absent": ("POST", "/absence/mark_absent"),
    "absence.mark_present": ("POST", "/absence/mark_present"),
    "absence.get_status": ("GET", "/absence/status/{employee_id}"),

    "sow.start": ("POST", "/sow/start"),
    "sow.update": ("POST", "/sow/update"),
    "sow.generate": ("POST", "/sow/generate/{sow_id}"),
}

async def execute_tool(client: httpx.AsyncClient, tool_call: dict):

```



```

name = tool_call.get("name")
args = tool_call.get("args", {})
if name not in TOOL_MAP:
    return {"ok": False, "error": f"Unknown tool {name}"}

method, path = TOOL_MAP[name]

# Path templating (e.g., /sow/generate/{sow_id})
for k, v in list(args.items()):
    placeholder = "{" + k + "}"
    if placeholder in path:
        path = path.replace(placeholder, str(v))
        args.pop(k, None)

if method == "GET":
    r = await client.get(path, params=args)
else:
    r = await client.post(path, json=args)
return r.json()

@router.post("")
async def orchestrate(req: ChatRequest) -> ChatResponse:
    snap = get_snapshot(req.session_id)

    # Optional domain nudge prior to LLM (cheap deterministic bias)
    snap["mode"] = route_domain(req.text, snap.get("mode", "IDLE"))

    # Ask Gemini for the strict envelope (decide intent + tool)
    plan: ChatEnvelope = ask_gemini(snap, req.text)

    # Execute tool if requested
    tool_result = None
    async with httpx.AsyncClient(base_url="http://localhost:8000") as client:
        if plan.tool_call:
            tool_result = await execute_tool(client,
plan.tool_call.model_dump())

    # Update session snapshot (FSM, active SOW id, collected slots, etc.)
    snap["mode"] = plan.mode
    if plan.intent == "start_sow" and tool_result and tool_result.get("ok"):
        snap["active_sow_id"] = tool_result.get("sow_id")
    snap["slots"] = {**snap.get("slots", {}), **(plan.slots or {})}
    save_snapshot(req.session_id, snap)

    # Reflect tool result in the message to user (lightly)
    message = plan.message_to_user
    if tool_result:
        if tool_result.get("ok") and tool_result.get("message"):

```

```

        message += f"\n\n✅ {tool_result['message']}"
    elif tool_result.get("url"):
        message += f"\n\n📄 Download: {tool_result['url']}"
    elif tool_result.get("error"):
        message += f"\n\n⚠️ {tool_result['error']}"

    return ChatResponse(
        mode=plan.mode,
        intent=plan.intent,
        slots=plan.slots,
        tool_call=plan.tool_call,
        message_to_user=message,
    )

```

backend/app/main.py

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.tools.absence import router as absence_router
from app.tools.sow import router as sow_router
from app.orchestrator import router as orchestrator_router

app = FastAPI(title="AI Orchestrator")

# CORS for local Next.js
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(absence_router)
app.include_router(sow_router)
app.include_router(orchestrator_router)

```

backend/requirements.txt

```

fastapi==0.115.0
uvicorn[standard]==0.30.6
pydantic==2.8.2
httpx==0.27.0
google-generativeai==0.7.2

```

backend/.env.example

```
GEMINI_API_KEY=your_key_here
DATABASE_URL=sqlite:///./orchestrator.db
REDIS_URL=
HOST_ABSENCE_INTERNAL=http://localhost:8000
HOST_SOW_INTERNAL=http://localhost:8000
```

Frontend (Next.js App Router)

frontend/app/api/chat/route.ts

```
import { NextRequest, NextResponse } from "next/server";

export async function POST(req: NextRequest) {
  const body = await req.json();
  const { session_id, text } = body;

  const r = await fetch("http://localhost:8000/orchestrator", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ session_id, text }),
  });

  const data = await r.json();
  return NextResponse.json(data);
}
```

frontend/app/page.tsx

```
"use client";
import { useState } from "react";

export default function Page() {
  const [sessionId] = useState(() => crypto.randomUUID());
  const [mode, setMode] = useState("IDLE");
  const [messages, setMessages] = useState<{ role: "user"|"ai"; text: string }
[]>([]);
  const [input, setInput] = useState("");

  async function send() {
    const text = input.trim();
    if (!text) return;
  }
}
```

```

setInput("");
setMessages(m => [...m, { role: "user", text }]);

const r = await fetch("/api/chat", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ session_id: sessionId, text }),
});
const data = await r.json();
setMode(data.mode);
setMessages(m => [...m, { role: "ai", text: data.message_to_user }]);
}

return (
  <div className="flex flex-col h-screen">
    <div className="p-3 border-b flex items-center gap-2">
      <span className="font-semibold">Session:</span>
      <span className="px-2 py-1 rounded bg-gray-100 border">{mode}</span>
    </div>
    <div className="flex-1 overflow-y-auto p-4 space-y-2">
      {messages.map((m, i) => (
        <div key={i} className={m.role === "user" ? "text-right" : "text-left">
          <div className={`inline-block px-3 py-2 rounded-lg ${m.role ===
"user" ? "bg-blue-200" : "bg-green-200"}`} >
            {m.text}
          </div>
        </div>
      ))}
    </div>
    <div className="p-3 border-t flex gap-2">
      <input
        className="flex-1 border rounded px-2"
        value={input}
        onChange={(e) => setInput(e.target.value)}
        onKeyDown={(e) => e.key === "Enter" && send()}
        placeholder="Type a message..."
      />
      <button className="px-4 py-2 rounded bg-blue-600 text-white"
onClick={send}>Send</button>
    </div>
  </div>
);
}

```

frontend/package.json

```
{
  "name": "gemini-orchestrator-ui",
  "private": true,
  "scripts": {
    "dev": "next dev -p 3000",
    "build": "next build",
    "start": "next start -p 3000"
  },
  "dependencies": {
    "next": "14.2.5",
    "react": "18.3.1",
    "react-dom": "18.3.1"
  },
  "devDependencies": {
    "typescript": "5.5.4"
  }
}
```

frontend/next.config.mjs

```
/** @type {import('next').NextConfig} */
const nextConfig = {};
export default nextConfig;
```

frontend/tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "forceConsistentCasingInFileNames": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true
  }
}
```

```
},  
"include": ["next-env.d.ts", "**/*.ts", "**/*.tsx"],  
"exclude": ["node_modules"]  
}
```

Why this is “best”

- **Strict JSON schema** enforced by Gemini (response_schema) ⇒ deterministic planning.
- **FSM snapshot** (mode + active SOW id + slots) ⇒ the agent stays inside the workflow.
- **Tool isolation** ⇒ LLM can only act through whitelisted endpoints.
- **Deterministic router** prior to LLM ⇒ fast/cheap bias toward correct domain.
- **Session store** (swap to Redis/DB) ⇒ persistence, multi-tab safety.
- **Simple, testable, extensible** ⇒ add tools or slots without refactoring core.

Optional: you can enable Gemini **native function-calling** later; the envelope-first pattern remains the most stable across SDK changes and is easier to audit.

Next Steps

1) Put your **real DB logic** into `tools/absence.py` and `tools/sow.py`. 2) If your two zips are separate services, point `httpx.AsyncClient(base_url=...)` to those hosts. 3) Replace the naive employee resolver with your actual employees table/search. 4) Add DOCX/PDF generation in `/sow/generate` using your fixed template. 5) Swap `state.py` with Redis (same interface) for multi-instance scaling.

Happy shipping! 🚚